

ADR 01 — Adoção de Arquitetura Desacoplada (Frontend Estático e Backend API REST)

- **Título:** ADR 01 — Desacoplamento Estrito entre Frontend (HTML/JS Puro) e Backend (API REST em Laravel 12)
- **Status:** Aprovado
- **Contexto:** O projeto necessita que a equipe desenvolva funcionalidades de forma paralela e independente, com clara divisão de responsabilidades entre interface e regras de negócio. Além disso, a visão de produto prevê a futura distribuição da plataforma para dispositivos móveis (via PWA/TWA) sem a necessidade de reescrever o backend ou mantê-lo acoplado a templates renderizados no servidor.
- **Decisão:** Separar a solução em dois repositórios isolados:
 - **radar-editais-backend:** Projeto API-STATELESS em Laravel 12 atuando exclusivamente como API REST, servindo dados estruturados em formato JSON a partir das rotas em `routes/api.php`.
 - **radar-editais-frontend:** Aplicação cliente estática construída em HTML5, CSS3 (Tailwind CSS) e JavaScript puro (Vanilla), comunicando-se com a API de forma assíncrona via `fetch()`.
- **Consequências:**
 - **Positivas (+):**
 - Independência total entre a camada visual e a lógica de negócios.
 - Reutilização da mesma API para clientes web, futuros aplicativos móveis e integrações externas. Deploys e versionamento independentes para frontend e backend.
 - **Negativas (-):**
 - Necessidade de configuração e gestão manual de políticas de CORS (`config/cors.php`) para liberar requisições entre origens distintas.
 - Gerenciamento manual de autenticação por *Bearer Tokens* (via Laravel Sanctum) armazenados no `localStorage` do navegador.

ADR 02 — Utilização do Framework Roach PHP para Mineração de Editais (ETL)

- **Título:** ADR 02 — Seleção do Framework Roach PHP para Coleta e Indexação Automatizada de Editais Públicos
- **Status:** Aprovado
- **Contexto:** Os editais de fomento à inovação (como FINEP, FAPESP e FAPPR) estão dispersos em portais governamentais com layouts instáveis, marcações HTML heterogêneas e tabelas complexas. O sistema exige uma esteira automatizada para raspar, higienizar, evitar duplicidades (idempotência) e persistir os dados dos editais

de forma padronizada **Decisão:** Adotar o framework [roach-php/laravel](#) associado ao [Symfony DomCrawler](#), estruturando os scrapers em classes *Spiders* isoladas por portal e orquestrando o tratamento de dados através de *Item Processors* no backend.

- **Consequências:**

- **Positivas (+):**

- Estruturação padronizada e modular das rotinas de raspagem em classes dedicadas por fonte.
 - Suporte nativo a pipelines de higienização de texto e prevenção de duplicidades via geração de chave única ([chave_unica](#)) [cite: 1, 3].
 - Facilidade de agendamento e execução em segundo plano através de comandos Artisan.

- **Negativas (-):**

- Exigência de configuração manual do *Service Provider* no ciclo de bootstrap do Laravel 12 ([bootstrap/providers.php](#)).
 - Necessidade de manutenção e atualização contínua dos seletores CSS/DOM sempre que as fontes governamentais alterarem seus layouts.